

Fracto - Doctor Appointment Booking System

Submitted by:

Hasvvath.R

Date: 26/03/2026

Batch Name: WIPRO NGA - .Net Full Stack

Angular - FY26

Instructor Name: Mr. Ramesh Nediyaadath

TABLE OF CONTENTS

S.No	Contents
1.	Problem Definition and Objectives
2.	Project Overview
3.	Technology Stack
4.	Frontend and Backend Architecture
5.	System Design
6.	Component Breakdown and API Design
7.	Database Design and Storage Optimization
8.	Entity Relationship Diagram (ERD)
9.	Major Functionalities Implemented
10.	Security Implementation (JWT Authentication)
11.	Advanced Features
12.	Conclusion

1. Problem Definition and Objectives

Problem Definition

Healthcare appointment scheduling is often inefficient due to manual booking systems or limited digital infrastructure. Patients frequently face challenges such as difficulty finding available doctors, long waiting times, and lack of centralized platforms for managing appointments. Hospitals and clinics also struggle to efficiently manage patient bookings, track appointment statuses, and maintain organized doctor schedules.

To address these challenges, the Fracto – Online Doctor Appointment Booking System was developed. The system provides a digital platform that allows users to search for doctors, filter them by city or specialization, book appointments based on available time slots, and manage appointments easily. Additionally, administrators can manage doctors, users, and appointments through an admin dashboard.

The system ensures that appointments are booked in a structured manner by enforcing slot restrictions, preventing duplicate bookings, and providing real-time doctor availability.

Objectives of the Project

The primary objectives of the Fracto system are:

- To provide a user-friendly platform for patients to search and book doctor appointments online.
- To enable users to filter doctors based on city, specialization, and ratings.
- To allow administrators to manage doctor records, patient data, and appointment schedules.
- To implement a secure authentication mechanism using JWT (JSON Web Token).
- To prevent duplicate bookings and manage time slot capacities effectively.
- To build a scalable full-stack application using Angular for the frontend and ASP.NET Core Web API for the backend.

To store and manage all system data efficiently using SQL Server database with Entity Framework Core ORM.

2. Project Overview

The Fracto Online Doctor Appointment Booking System is a full-stack web application developed to streamline the doctor appointment scheduling process. The application allows patients to register, log in, browse available doctors, and book appointments based on available time slots. The system also includes an administrative module for managing doctors, users, and appointment statuses.

The application consists of three major layers:

1. Frontend Layer (Angular)
2. Responsible for displaying the user interface and handling user interactions.
3. Backend Layer (ASP.NET Core Web API)
4. Responsible for implementing business logic, authentication, and database operations.
5. Database Layer (SQL Server)

Responsible for storing persistent data including users, doctors, appointments, and ratings.

The application ensures secure communication between the frontend and backend through REST APIs and JWT-based authentication.

3. Technology Stack

The following technologies were used to implement the system:

Frontend

The frontend of the application is developed using Angular, a powerful TypeScript-based framework used for building modern web applications. Angular provides a component-based architecture that enables modular development and improved maintainability.

Key frontend technologies include:

- Angular Framework
- TypeScript
- HTML
- CSS
- Axios for API communication
- NgRx Store for state management
- Backend

The backend is implemented using ASP.NET Core Web API, which provides a robust platform for building scalable and secure REST APIs.

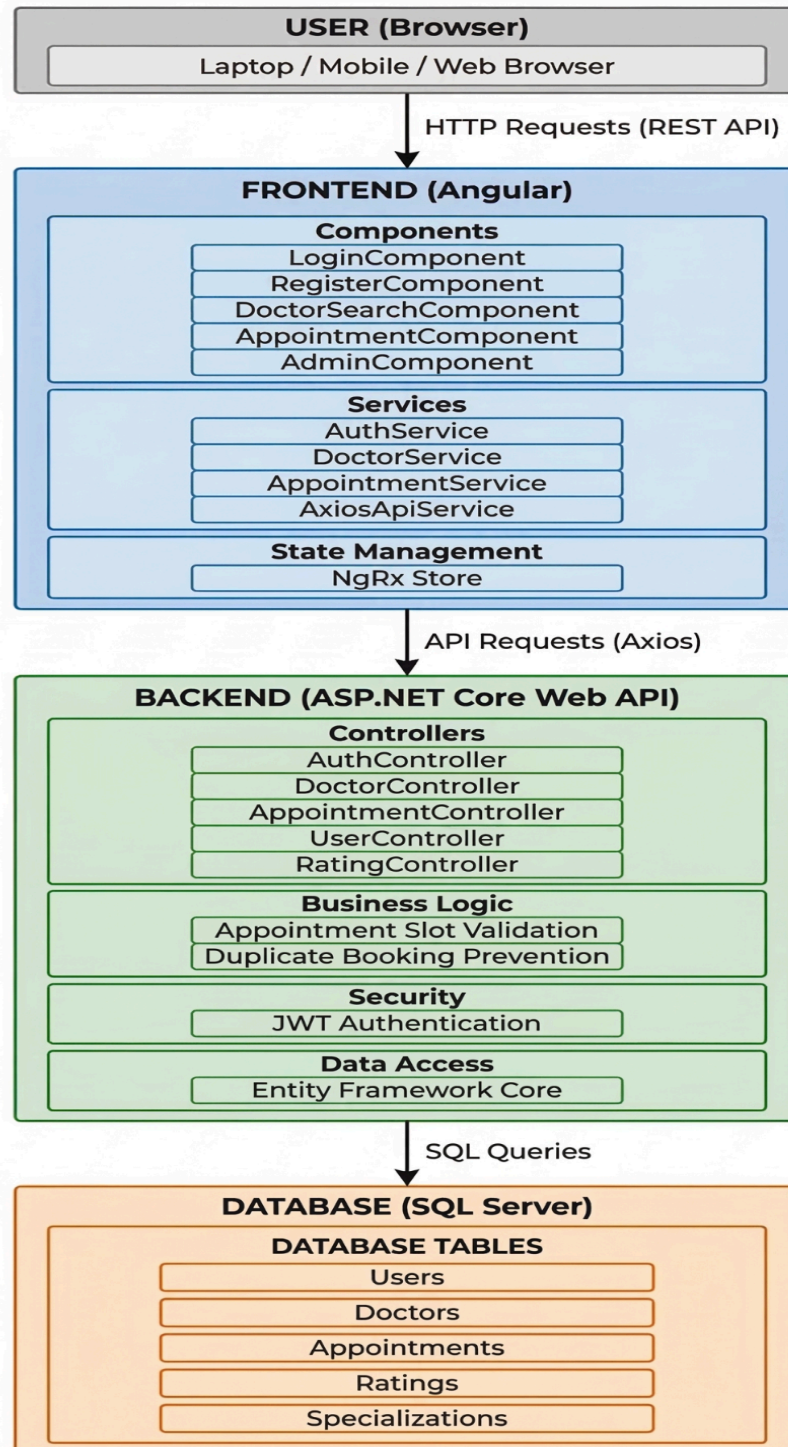
Key backend technologies include:

- ASP.NET Core Web API
- Entity Framework Core
- JWT Authentication
- Swagger for API testing
- Database

The system uses Microsoft SQL Server as the primary database. Entity Framework Core is used as the ORM (Object Relational Mapper) to map C# classes to database tables.

4. Frontend and Backend Architecture

The system follows a three-tier architecture which separates concerns and improves scalability.



The Angular frontend follows a modular architecture consisting of components and services.

Key components include:

- Login Component
- Register Component
- Doctor Search Component
- Appointment Booking Component
- Admin Dashboard Component

Each component handles a specific functionality within the system.

Angular services are used to manage API communication and reusable business logic.

Examples include:

- AuthService
- DoctorService
- AppointmentService
- AxiosApiService
- Backend Architecture
- The backend follows a RESTful architecture using ASP.NET Core controllers.

Major controllers include:

- AuthController
- DoctorController
- AppointmentController
- RatingController
- UserController

Each controller exposes API endpoints that perform CRUD operations on the system data.

Entity Framework Core is used to interact with the SQL Server database through the ApplicationDbContext class.

5. System Design

The system design focuses on providing a smooth interaction between the user interface and backend services.

Users interact with the Angular frontend, which sends HTTP requests to backend API endpoints. The backend processes the request, interacts with the database, and sends responses back to the frontend.

Example workflow:

- User logs in through Angular.
- Angular sends a login request to the backend API.
- Backend validates credentials and generates a JWT token.
- Token is stored in the browser and used for subsequent API requests.

6. Component Breakdown and API Design

Frontend Components

The frontend consists of multiple Angular components responsible for specific functionalities.

Login Component

Handles user authentication by accepting user credentials and sending login requests to the backend API.

Doctor Search Component

Displays a list of available doctors and allows filtering based on city, specialization, and ratings.

Appointment Component

Allows users to select doctors, choose dates, and book available time slots.

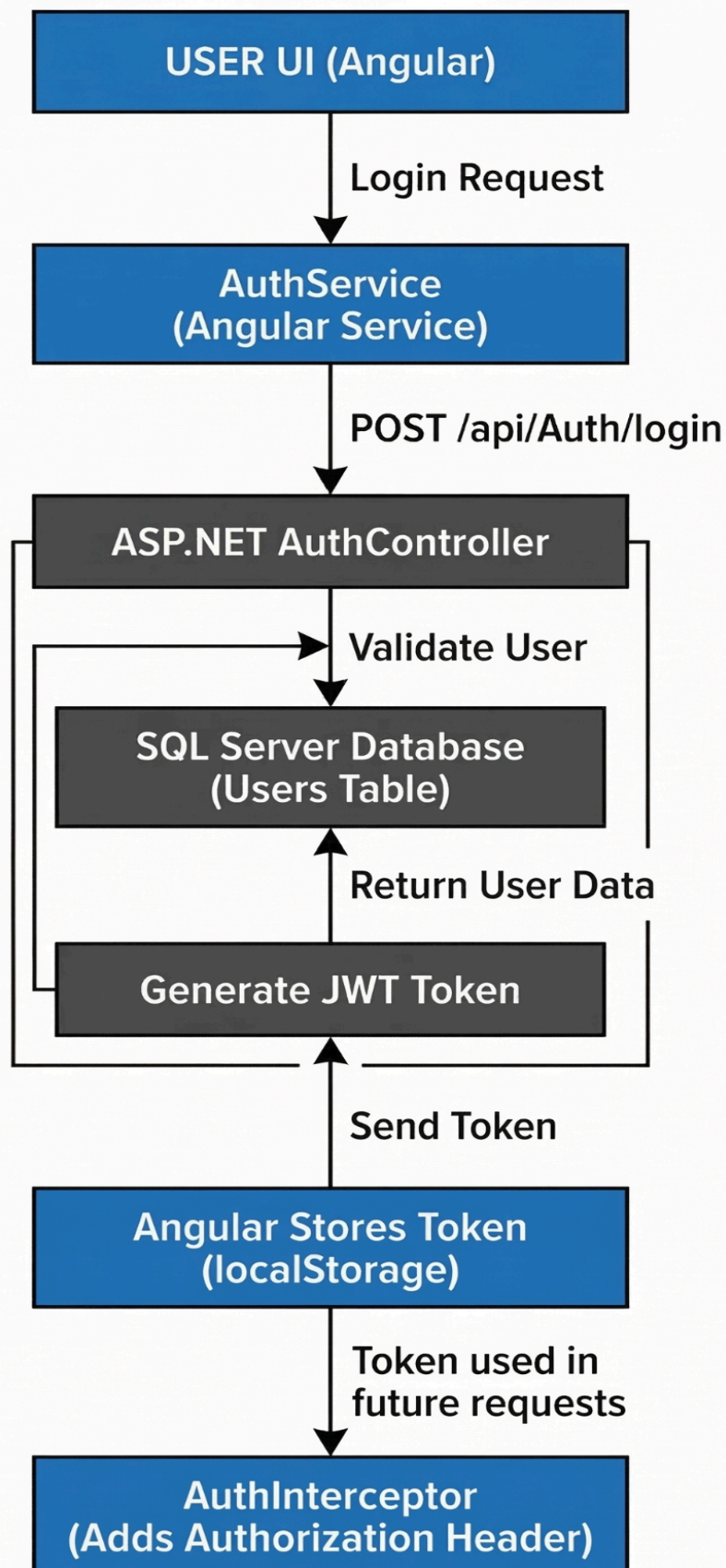
Admin Component

Provides administrative capabilities such as managing doctors, users, and appointments.

API Endpoints

Key API endpoints implemented in the backend include:

Category	Method	Endpoint
Authentication APIs	POST ▾	/api/Auth/register ▾
	POST ▾	/api/Auth/login ▾
Doctor APIs	GET ▾	/api/Doctor ▾
	POST ▾	/api/Doctor ▾
	PUT ▾	/api/Doctor/{id} ▾
	DELETE ▾	/api/Doctor/{id} ▾
Appointment APIs	POST ▾	/api/Appointment/b... ▾
	GET ▾	/api/Appointment ▾
	PUT ▾	/api/Appointment/{...} ▾
	DELETE ▾	/api/Appointment/{...} ▾
User APIs	GET ▾	/api/User ▾
	PUT ▾	/api/User/{id} ▾
	DELETE ▾	/api/User/{id} ▾



7. Database Design and Storage Optimization

The system database is designed to efficiently store user accounts, doctor information, and appointment records.

The main database tables include:

- Users
- Doctors
- Appointments
- Ratings
- Specializations

To optimize database performance:

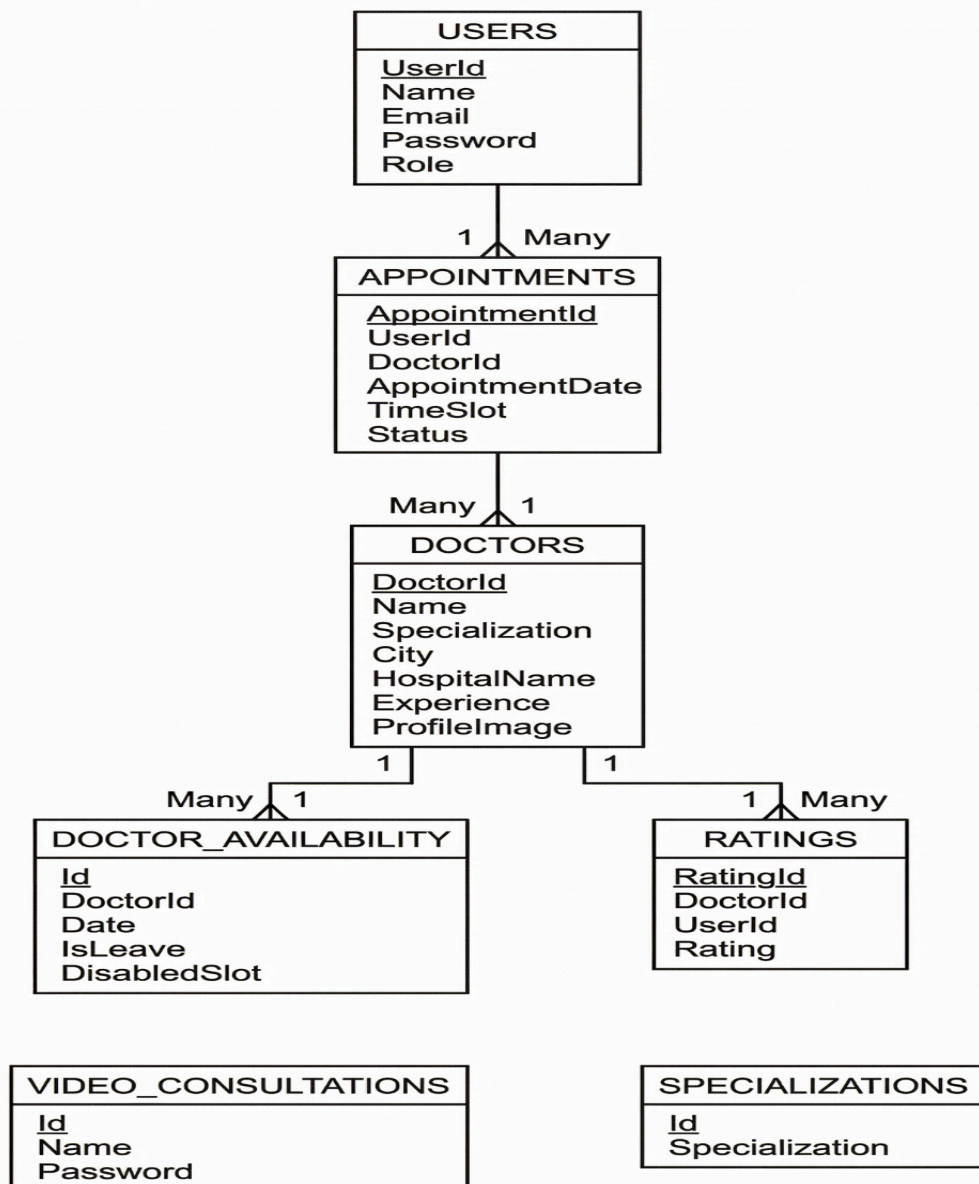
- Unique indexing is applied to user emails to prevent duplicates.
- Relationships between tables reduce redundancy.
- Foreign keys maintain data consistency.

8. Entity Relationship Diagram (ERD)

The ERD defines the relationships between entities in the database.

Key relationships include:

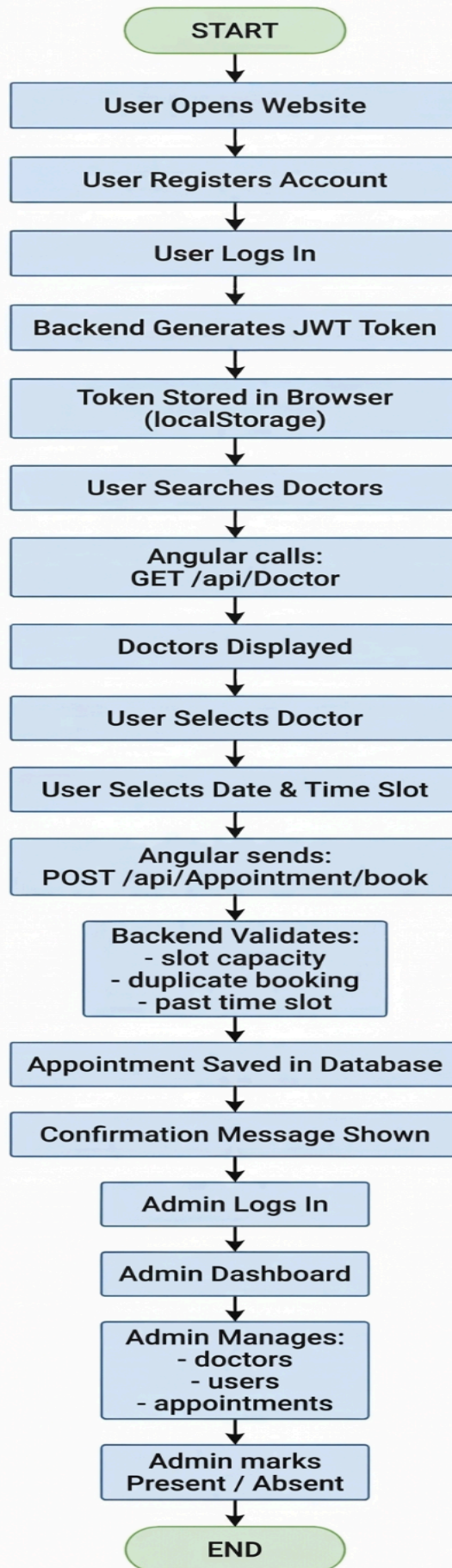
- One User can book multiple Appointments.
- One Doctor can have multiple Appointments.
- One Doctor can receive multiple Ratings.



9. Major Functionalities Implemented

The application provides the following major functionalities.

- User Features
 - User registration
 - User login
 - Doctor search
 - Filtering doctors by city, specialization, and rating
 - Appointment booking
 - Viewing appointments
- Admin Features
 - Add doctor
 - Edit doctor details
 - Delete doctor
 - View registered users
 - Manage appointments
 - Mark appointments as present or absent



10. Security Implementation (JWT Authentication)

JWT authentication is used to secure API endpoints.

When a user logs in successfully:

- The backend generates a JWT token.
- The token contains user information such as user ID and role.
- The frontend stores the token in localStorage.
- The token is attached to every API request through an Angular interceptor.

The backend verifies the token before allowing access to protected endpoints.

This ensures that only authenticated users can perform sensitive operations.

11. Advanced Features

In Sprint 2, the system was enhanced with advanced real-world features to improve functionality, scalability, and user experience. These features extend the system beyond basic CRUD operations and introduce real-time communication, remote consultation, and dynamic scheduling.

1. SignalR Integration (Real-Time Communication)

SignalR was implemented to enable real-time communication between the frontend and backend. Unlike traditional HTTP requests, SignalR allows bidirectional communication using WebSockets.

This feature ensures that updates such as appointment status changes (Present/Absent) are reflected instantly without requiring a page refresh.

Benefits:

- Real-time updates
- Improved user experience
- Reduced manual refresh operations

2. Video Consultation Feature

A video consultation feature was introduced to enable remote interaction between patients and doctors. This feature allows users to consult doctors online without visiting physically.

The system uses real-time communication technologies along with SignalR for connection signaling.

Workflow:

- User books an appointment
- User joins video consultation session
- SignalR establishes connection between doctor and patient

- Video communication takes place

Benefits:

- Supports telemedicine
- Enables remote healthcare access
- Reduces dependency on physical visits

3. Doctor Availability and Calendar Management

A doctor calendar system was implemented to manage doctor availability. Doctors can:

- Mark leave days
- Disable specific time slots

A new data structure is introduced to store availability information.

This prevents users from booking appointments during unavailable periods.

Benefits:

- Prevents invalid bookings
- Improves scheduling accuracy
- Reflects real-world doctor availability

4. Doctor Experience Enhancement

An experience field was added to the doctor profile to provide additional information about the doctor's expertise.

Example:

- Doctor Name
- Specialization
- Experience (e.g., 10 years)

This helps users make better decisions while selecting doctors.

Benefits:

- Improves user trust
- Enhances profile completeness
- Supports better filtering in future

5. System Improvement Summary

The Sprint 2 enhancements significantly improve the system by introducing:

- Real-time updates using SignalR
- Remote video consultation capability
- Dynamic doctor availability management
- Enhanced doctor profile information

These features make the system closer to a real-world healthcare application and improve usability, scalability, and functionality.

12. Conclusion

The Fracto Online Doctor Appointment Booking System successfully demonstrates the development of a full-stack web application integrating an Angular frontend, ASP.NET Core Web API backend, and SQL Server database. The system provides a scalable and secure platform for managing doctor appointments, offering user-friendly features for patients and efficient administrative tools for system management.

The implementation of JWT authentication ensures secure access control, while Entity Framework Core simplifies database interactions and improves database management. The modular architecture of Angular components and services enables maintainable, scalable, and efficient frontend development.

Overall, the project demonstrates the practical implementation of modern full-stack development concepts including RESTful APIs, secure authentication, state management, and structured database design.

The system has been further enhanced by implementing doctor availability management, video consultation, and real-time communication using SignalR, making it closer to real-world healthcare applications.